

HOSPITAL APPOINTMENT BOOKING SYSTEM
REST API, AUTHENTICATION & WEB APPLICATION
MediCare

Full-stack Hospital Appointment Booking System implemented according to the project PS.
Prepared using the uploaded Millbrook Apiary report as the formatting/reference model.

CONTENTS

1. Abstract
2. Problem Statement and Objectives
3. Project Scope and Requirements
4. Technology Stack and Tools
5. System Architecture
6. Database Design
7. Backend Implementation
8. Authentication and Authorization
9. Appointment Booking and Slot Conflict Validation
10. Doctor Appointment Schedule
11. Frontend Implementation
12. Complete User Workflow
13. PS Objective Mapping
14. API Endpoints
15. Testing and Verification
16. Security and Error Handling
17. Limitations and Future Scope
18. Conclusion

Appendix A — Implementation Screenshots

1. ABSTRACT

The Hospital Appointment Booking System is a full-stack web application developed to allow patients to register, authenticate, view available doctors and book appointments for available time slots. The system also provides a separate doctor portal through which authenticated doctors can log in and view their scheduled appointments.

The backend uses Node.js and Express.js to expose REST APIs, MongoDB to persist data and Mongoose to define schemas and references. JWT provides authentication, while bcryptjs protects passwords through hashing. The frontend is built with React and Vite and communicates with the backend using Axios.

The appointment process validates the selected doctor, verifies that the selected time is an available slot and prevents the same doctor/date/time combination from being booked twice. This directly addresses the core booking and validation requirements in the PS.

Main Outcomes

Patient registration and JWT login

Protected appointment booking

Mongoose Patient, Doctor and Appointment schemas

Doctor authentication and schedule API

React patient and doctor portals

2. PROBLEM STATEMENT AND OBJECTIVES

The PS requires a Hospital Appointment Booking System that manages patients, doctors and appointments using Mongoose. It requires patient registration and JWT-based login, slot-conflict validation before booking, authentication middleware for protected booking routes and an API for doctors to view their scheduled appointments.

A separate doctor login was added only to support objective 5 properly. It is not an admin system and does not introduce unrelated functionality.

3. PROJECT SCOPE AND REQUIREMENTS

Functional Scope

Patient registration, login and logout.

Doctor records with specialization and available appointment slots.

Doctor login and authenticated doctor session.

Appointment booking with date and time.

Validation of doctor existence and available slots.

Duplicate doctor/date/time booking prevention.

Patient appointment history.

Doctor-specific scheduled appointment view.

Responsive React interface for the patient and doctor portals.

Features intentionally not added

To keep the implementation aligned with the PS, features such as Firebase, Socket.io, payments, GPS, notifications, file uploads, Swagger, analytics and a complex admin/role system were not added.

Project Deliverables

Express REST API

MongoDB/Mongoose database

JWT + bcryptjs authentication

React frontend

Thunder Client API testing

.env configuration

Project documentation with screenshots

4. TECHNOLOGY STACK AND TOOLS

Backend Configuration

The backend runs on port 5001 and connects to the hospital_appointment MongoDB database. Environment variables are used for PORT, MONGO_URI and JWT_SECRET.

5. SYSTEM ARCHITECTURE

The system follows a simple full-stack architecture: React frontend → Express REST API → MongoDB database. Authentication middleware and Mongoose models sit inside the backend layer.

Figure 1. System architecture of the Hospital Appointment Booking System.

Request Flow

React collects user input.

Axios sends an HTTP request to Express.

Express selects the appropriate route.

Authentication middleware verifies JWT for protected requests.

Mongoose reads or writes the relevant MongoDB collection.

Express returns JSON.

React updates the page based on the response.

6. DATABASE DESIGN

The database is named `hospital_appointment` and contains `patients`, `doctors` and `appointments` collections.

Patient

Doctor

Appointment

The Appointment schema uses Mongoose references so patient and doctor data can be populated when appointments are retrieved. A unique compound index on doctor, date and time prevents duplicate booking records at the database level.

7. BACKEND IMPLEMENTATION

The backend is separated into models, routes and middleware so each part of the system has a clear responsibility.

Figure 2. Thunder Client evidence of doctor API creation during development.

8. AUTHENTICATION AND AUTHORIZATION

Patient Authentication

During registration, `bcryptjs` hashes the password before the Patient document is created. During login, `bcrypt.compare` checks the entered password against the stored hash. A successful login creates a JWT containing the patient ID and email.

Patient Middleware

`authMiddleware` reads the Authorization header, checks for the Bearer format, verifies the token using `JWT_SECRET` and stores the decoded identity in `req.patient`. Invalid or expired tokens receive HTTP 401.

Doctor Authentication

Doctor login uses `/api/doctor-auth/login`. The returned JWT includes the doctor ID, email and role: `doctor`. `doctorAuthMiddleware` verifies the token and rejects tokens whose role is not `doctor`.

Figure 3. Successful doctor authentication response. JWT token value redacted for report safety.

Why JWT and bcryptjs?

JWT gives protected REST endpoints a verifiable user identity.

`bcryptjs` prevents passwords from being stored in plain text.

Separate doctor middleware ensures doctor-only schedule access.

9. APPOINTMENT BOOKING AND SLOT CONFLICT VALIDATION

The patient chooses a doctor, date and time in the React booking interface. The protected POST /api/appointments endpoint receives doctor, date and time and uses the patient identity from the JWT.

Database Protection

The route checks for an existing appointment before insertion, while the Appointment model also defines a unique compound index on doctor + date + time. Together these provide application-level and database-level protection.

Figure 4. Patient UI showing successful appointment booking.

10. DOCTOR APPOINTMENT SCHEDULE

The doctor schedule requirement is implemented through GET /api/appointments/doctor/my. The route does not accept an arbitrary doctor ID for normal access; it uses the doctor identity contained in the authenticated JWT.

Doctor logs in and receives a JWT.

Doctor token is sent as Authorization: Bearer .

doctorAuthMiddleware verifies the JWT and role.

req.doctor.id identifies the logged-in doctor.

Appointment.find({ doctor: req.doctor.id }) retrieves only that doctor's appointments.

Patient and doctor information are populated with Mongoose.

The result is returned as JSON for the Doctor Dashboard.

Why this matches the PS

This directly demonstrates that an authenticated doctor can view their own scheduled appointments. It also avoids exposing one doctor's schedule to another user.

11. FRONTEND IMPLEMENTATION

The React application uses a MediCare design with separate patient and doctor entry points. The frontend stores the appropriate JWT and uses Axios headers for protected requests.

Patient Portal

Landing page and patient login.

Patient registration.

Doctor listing with specialization and available slots.

Booking modal with date and time selection.

Booking confirmation message.

My Appointments section.

Logout.

Figure 5. MediCare landing page with patient and doctor access.

Figure 6. Patient dashboard showing a confirmed appointment.

11.1 DOCTOR PORTAL

The doctor portal provides a separate login and dashboard. The doctor dashboard requests the authenticated doctor's schedule from the protected doctor schedule API. This creates a complete patient-to-doctor appointment workflow rather than exposing schedule data on the patient screen.

UI Design

The interface uses medical green colors, card-based doctor listings, confirmation badges, clear forms, responsive grids and a booking modal. The styling is maintained in App.css while application logic is handled in App.jsx.

Figure 7. Doctor selection cards used by the patient portal.

12. COMPLETE USER WORKFLOW

Patient Workflow

Open MediCare.
Register or log in as a patient.
Receive JWT after successful login.
View doctors from MongoDB.
Select a doctor and available slot.
Send protected booking request.
Backend validates doctor, slot and conflict.
Appointment is saved in MongoDB.
Patient dashboard displays the appointment.

Doctor Workflow

Open Doctor Login.
Enter doctor email and password.
Server verifies password using bcryptjs.
Server creates doctor JWT with role=doctor.
Doctor dashboard requests /api/appointments/doctor/my.
Middleware verifies the doctor token.
Only that doctor's appointments are returned.
Doctor views patient, date and time information.

13. PS OBJECTIVE MAPPING

PS Deliverable Demonstration

For a viva or demonstration, the project can be shown in the following order: database schemas → patient registration/login → JWT → doctor listing → appointment booking → slot conflict validation → doctor login → doctor schedule API → React doctor dashboard.

14. API ENDPOINTS

Typical Protected Request

The frontend sends the token using the HTTP Authorization header in the form: Authorization: Bearer . The middleware verifies the token before the route handler continues.

Important HTTP Responses

15. TESTING AND VERIFICATION

During development, the React patient interface displayed a successful appointment booking confirmation and the My Appointments section displayed the booked appointment. Doctor login was verified through Thunder Client with a successful 200 response and JWT.

Database Verification

MongoDB Compass was used throughout development to inspect the hospital_appointment database, doctors, patients and appointments collections. Duplicate doctor test data was cleaned before the final doctor account setup.

16. SECURITY AND ERROR HANDLING

Security

Passwords are hashed using bcryptjs.

JWT secret and MongoDB URI are kept in .env.

Protected routes require Bearer JWT authentication.

Doctor schedule access verifies role=doctor.

Doctor listing responses exclude password fields.

Appointments store references instead of duplicating full user documents.

Error Handling

400 for missing input, unavailable slot or booking conflict.

401 for missing, invalid or expired authentication.

403 for non-doctor access to doctor-only routes.

404 for an unknown doctor.

500 for unexpected backend/database failures.

Middleware Flow

Request → CORS/JSON parsing → authentication middleware when required → route handler → Mongoose query/write → JSON response.

17. LIMITATIONS AND FUTURE SCOPE

Current Limitations

The project is intended for academic/local demonstration.

Doctor slots are predefined strings rather than a complete calendar engine.

Appointment cancellation and rescheduling are not included.

The demonstration password is simple and should be stronger in production.

There is no admin panel because it is outside the PS scope.

Future Scope

Doctor-managed availability.

Appointment cancellation/rescheduling.

Email/SMS notifications.

Production deployment.

Stronger password policy and refresh tokens.

Formal API documentation if later required.

These improvements are intentionally outside the current implementation so the project remains focused on the PS.

18. CONCLUSION

The Hospital Appointment Booking System successfully implements the major PS requirements using a focused full-stack architecture. Patients can register and authenticate, doctors and appointment slots are stored using Mongoose, and appointments connect patients and doctors through ObjectId references.

The appointment workflow validates doctor existence, available slots and duplicate bookings. JWT authentication and middleware protect the booking and appointment routes. A separate doctor authentication flow makes the scheduled-appointment API practical and secure because the logged-in doctor's identity is taken from the JWT.

The React frontend connects these backend capabilities to a professional MediCare interface. Thunder Client was used for API testing and MongoDB Compass was used to verify persistent data. The result is a clear academic demonstration of REST APIs, authentication, middleware, MongoDB/Mongoose relationships, validation and React-to-backend communication.

APPENDIX A — IMPLEMENTATION SCREENSHOTS

The screenshots below are authentic development screenshots from the project. They are included as evidence of the implementation and testing process.

Screenshot A1 — MediCare landing page.

Screenshot A2 — Patient dashboard with confirmed appointment.

Screenshot A3 — Appointment booking success.

Screenshot A4 — Thunder Client doctor creation API.

Screenshot A5 — Successful doctor authentication; JWT value redacted.

Screenshot A6 — Doctor selection and booking cards.

Recommended final evidence screenshot: capture the completed Doctor Dashboard showing the logged-in doctor and at least one scheduled appointment after the final UI/CSS test.

APPENDIX B — KEY FILES

Final Demonstration Checklist

Start MongoDB.

Run backend with npm run dev.

Run frontend with npm run dev.

Register/login as patient.

Book a doctor slot.

Show My Appointments.

Attempt a duplicate slot to demonstrate conflict validation.

Log in as a doctor.

Show the doctor schedule API and Doctor Dashboard.

Explain Patient, Doctor and Appointment Mongoose references.

Formatting reference: the uploaded Millbrook Apiary & Honey Cooperative report was used as the structural and visual reference for this document, including its cover, contents, sectioned report format, tables and screenshot appendix.

filecite turn1file0 L20-L41

APPENDIX C — UPDATED TESTING EVIDENCE

The following screenshots provide updated evidence of the backend APIs, authentication, appointment validation, database records, and React frontend. Sensitive passwords and JWT tokens have been redacted for documentation.

Figure C1 — Backend root endpoint verification (GET /).

Figure C2 — Doctors API returning the configured doctor records and available slots.

Figure C3 — Backend health endpoint verification (GET /health).

Figure C4 — Doctor creation API successfully creating a new doctor account.

Figure C5 — Patient login API returning a successful JWT authentication response (token redacted).

Figure C6 — Patient registration API successfully creating a patient account.

Figure C7 — Slot-conflict validation rejecting a duplicate doctor/date/time booking.

Figure C8 — Successful appointment booking response from the protected appointment API.

Figure C9 — Available-slot validation rejecting a time not offered by the selected doctor.

Figure C10 — Doctor login API returning successful doctor authentication (JWT redacted).

Figure C11 — Patient appointment API showing the authenticated patient's booked appointment (token redacted).

Figure C12 — Doctor schedule API showing the authenticated doctor's scheduled appointment (token redacted).

Figure C13 — MongoDB Compass view of the hospital_appointment doctors collection.

Figure C14 — MediCare patient dashboard showing available doctors and booking controls.

Figure C15 — MediCare patient dashboard showing the upcoming appointment after booking.

DETAIL	VALUE
Student	Aditya Khare
Program	B.Tech Computer Science and Engineering
Batch	2025–2029
Class / Cohort	Demis Hassabis
Semester	II
Frontend	React.js + Vite + Axios
Backend	Node.js + Express.js
Database	MongoDB + Mongoose
Authentication	JWT + bcryptjs

PS OBJECTIVE	REQUIREMENT	HOW THIS PROJECT USES IT
1	Patient, Doctor and Appointment schemas	Implemented as Mongoose models with ObjectId references.
2	Patient registration + JWT login	POST /api/auth/register and POST /api/auth/login.
3	Slot-conflict validation	Checks doctor availability and existing doctor/date/time booking; unique index adds database protection.
4	Authentication middleware	Bearer JWT required for protected patient appointment routes.
5	Doctor appointment schedule API	GET /api/appointments/doctor/my protected by doctor JWT middleware.

TECHNOLOGY	PURPOSE
React.js	Component-based frontend and portal interfaces.
Vite	React development environment.
Axios	Frontend-to-backend HTTP communication.
Node.js	Backend JavaScript runtime.
Express.js	REST API and routing framework.
MongoDB	Document database.
Mongoose	Schemas, models, references and database operations.
JWT	Patient and doctor authentication tokens.
bcryptjs	Password hashing and verification.

TECHNOLOGY	PURPOSE
dotenv	Environment configuration.
cors	Frontend/backend cross-origin communication.
Thunder Client	REST API testing.
MongoDB Compass	Database inspection and management.

FIELD	TYPE	PURPOSE
name	String	Patient name
email	String / unique	Login identity
password	String	bcryptjs hash
timestamps	Date	Creation/update time

FIELD	TYPE	PURPOSE
name	String	Doctor name
email	String / unique	Doctor login identity
password	String	bcryptjs hash
specialization	String	Medical specialization
availableSlots	[String]	Bookable times

FIELD	TYPE	PURPOSE
patient	ObjectId → Patient	Patient reference
doctor	ObjectId → Doctor	Doctor reference
date	String	Appointment date
time	String	Appointment time

FILE / FOLDER	RESPONSIBILITY
models/Patient.js	Patient schema and model
models/Doctor.js	Doctor schema and model
models/Appointment.js	Appointment schema, references and unique index
routes/authRoutes.js	Patient register/login
routes/doctorRoutes.js	Doctor listing and creation
routes/doctorAuthRoutes.js	Doctor login
routes/appointmentRoutes.js	Booking, patient appointments and doctor schedule
middleware/authMiddleware.js	Patient JWT verification
middleware/doctorAuthMiddleware.js	Doctor JWT + role verification
server.js	Express setup, routes, MongoDB connection and server startup

STEP	VALIDATION	OUTCOME
1	Patient token	Unauthenticated users cannot book.
2	Required fields	Doctor, date and time are mandatory.
3	Doctor lookup	Unknown doctor returns 404.

STEP	VALIDATION	OUTCOME
4	Available slot	Selected time must exist in availableSlots.
5	Existing appointment	Same doctor/date/time cannot already exist.
6	Create	Appointment stores patient and doctor ObjectIds.

FRONTEND ACTION	API	AUTHENTICATION
Register patient	POST /api/auth/register	Public
Login patient	POST /api/auth/login	Public
Load doctors	GET /api/doctors	Public
Book appointment	POST /api/appointments	Patient JWT
View patient appointments	GET /api/appointments/my	Patient JWT
Login doctor	POST /api/doctor-auth/login	Public
View doctor schedule	GET /api/appointments/doctor/my	Doctor JWT

PS REQUIREMENT	PROJECT FILE / API	IMPLEMENTATION
Patient schema	models/Patient.js	Mongoose schema with unique email and hashed password.
Doctor schema	models/Doctor.js	Mongoose schema with credentials, specialization and slots.
Appointment schema	models/Appointment.js	Mongoose references to Patient and Doctor.
Patient registration	POST /api/auth/register	Validates input, checks existing email and hashes password.
JWT login	POST /api/auth/login	bcrypt verification + JWT creation.
Slot conflict validation	POST /api/appointments	Doctor slot check + existing appointment check + unique index.
Authentication middleware	authMiddleware.js	Bearer JWT validation for protected patient routes.
Doctor schedule API	GET /api/appointments/doctor/my	Doctor JWT identity controls the appointment query.

METHOD	ENDPOINT	PURPOSE	AUTH
POST	/api/auth/register	Register patient	None
POST	/api/auth/login	Patient login	None
GET	/api/doctors	List doctors	None
POST	/api/doctors	Create doctor(s)	None / setup
POST	/api/doctor-auth/login	Doctor login	None
POST	/api/appointments	Book appointment	Patient JWT
GET	/api/appointments/my	Patient appointments	Patient JWT
GET	/api/appointments/doctor/my	Doctor schedule	Doctor JWT

STATUS	MEANING IN THIS PROJECT
200	Successful login or data retrieval
201	Patient/doctor/appointment created
400	Missing fields, unavailable slot or conflict
401	Authentication required or invalid token
403	Doctor role required
404	Doctor not found

STATUS	MEANING IN THIS PROJECT
500	Unexpected server/database error

TEST	ACTION	RESULT	STATUS
Backend startup	npm run dev	Server running on port 5001	Pass
MongoDB connection	Start backend	MongoDB connected successfully	Pass
Doctor creation	POST /api/doctors	Doctor records created	Pass
Doctor login	POST /api/doctor-auth/login	HTTP 200 + JWT	Pass
Patient booking	POST /api/appointments	Booking success response	Pass
Patient appointments	GET /api/appointments/my	Booked appointment returned	Pass
Doctor schedule	GET /api/appointments/doctor/my	Authenticated doctor's schedule	Implemented
Slot conflict	Repeat same doctor/date/time	Request rejected	Implemented
Invalid token	Protected route without valid JWT	HTTP 401	Implemented

FILE	ROLE
backend/server.js	Express application and MongoDB connection
backend/models/Patient.js	Patient Mongoose model
backend/models/Doctor.js	Doctor Mongoose model
backend/models/Appointment.js	Appointment model and conflict index
backend/routes/authRoutes.js	Patient registration/login
backend/routes/doctorRoutes.js	Doctor creation/listing
backend/routes/doctorAuthRoutes.js	Doctor login
backend/routes/appointmentRoutes.js	Booking and appointment APIs
backend/middleware/authMiddleware.js	Patient JWT middleware
backend/middleware/doctorAuthMiddleware.js	Doctor JWT middleware
frontend/src/App.jsx	React application and user flows
frontend/src/App.css	MediCare interface styling
frontend/src/main.jsx	React entry point
backend/.env	Local configuration values

APPENDIX — SCREENSHOT TESTING EVIDENCE

The following screenshots document the implemented API, authentication, appointment booking, conflict validation, doctor schedule, database, and frontend workflow.

MediCare — System Architecture

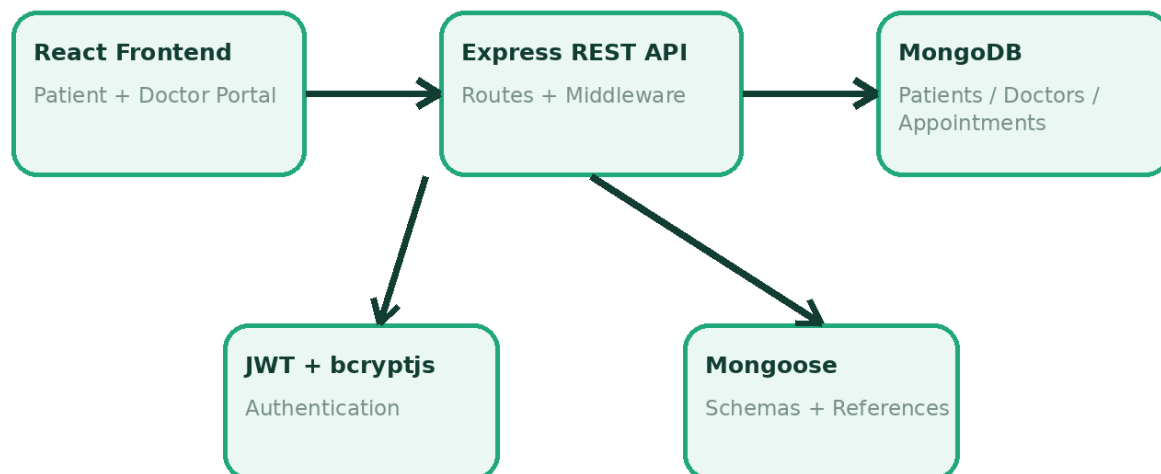


Figure 1: Project implementation and testing evidence.

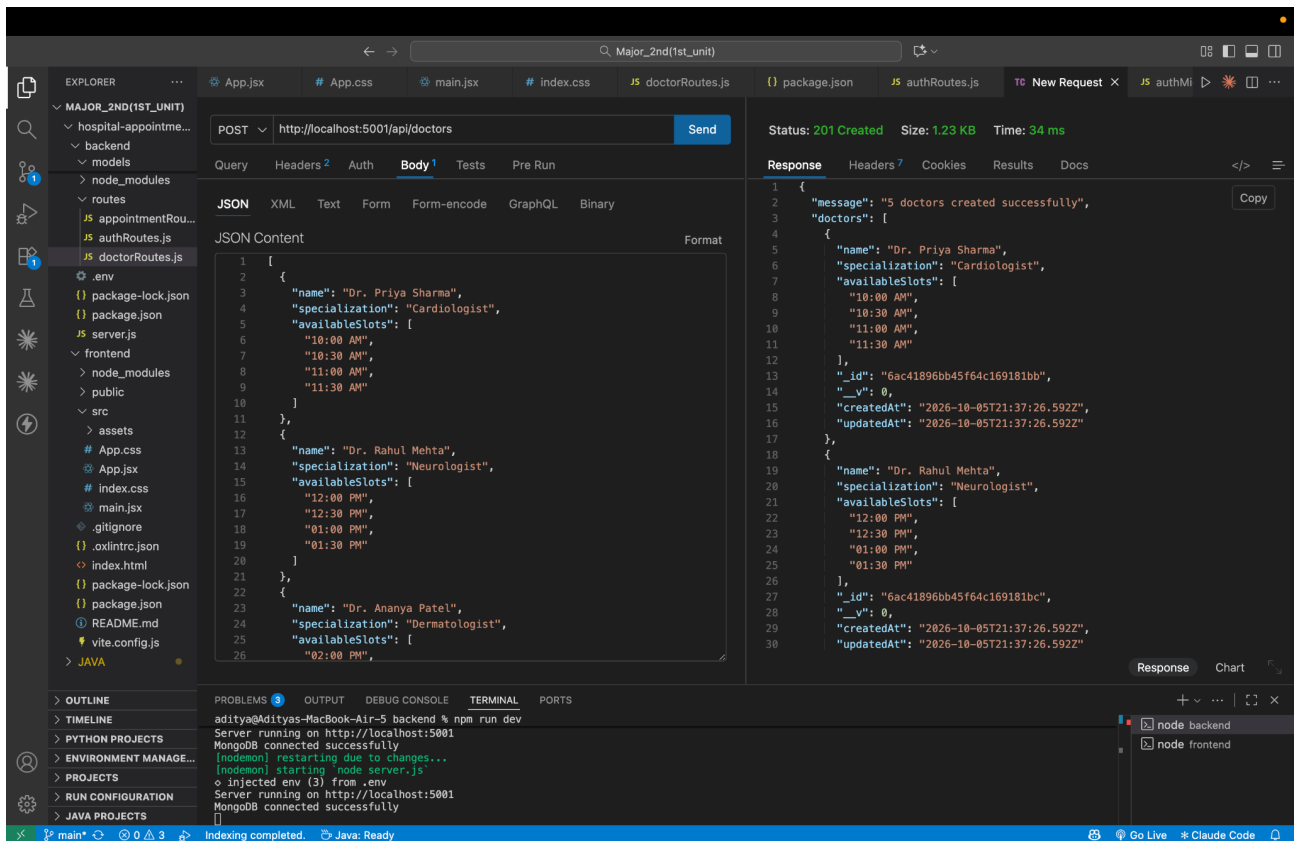


Figure 2: Project implementation and testing evidence.

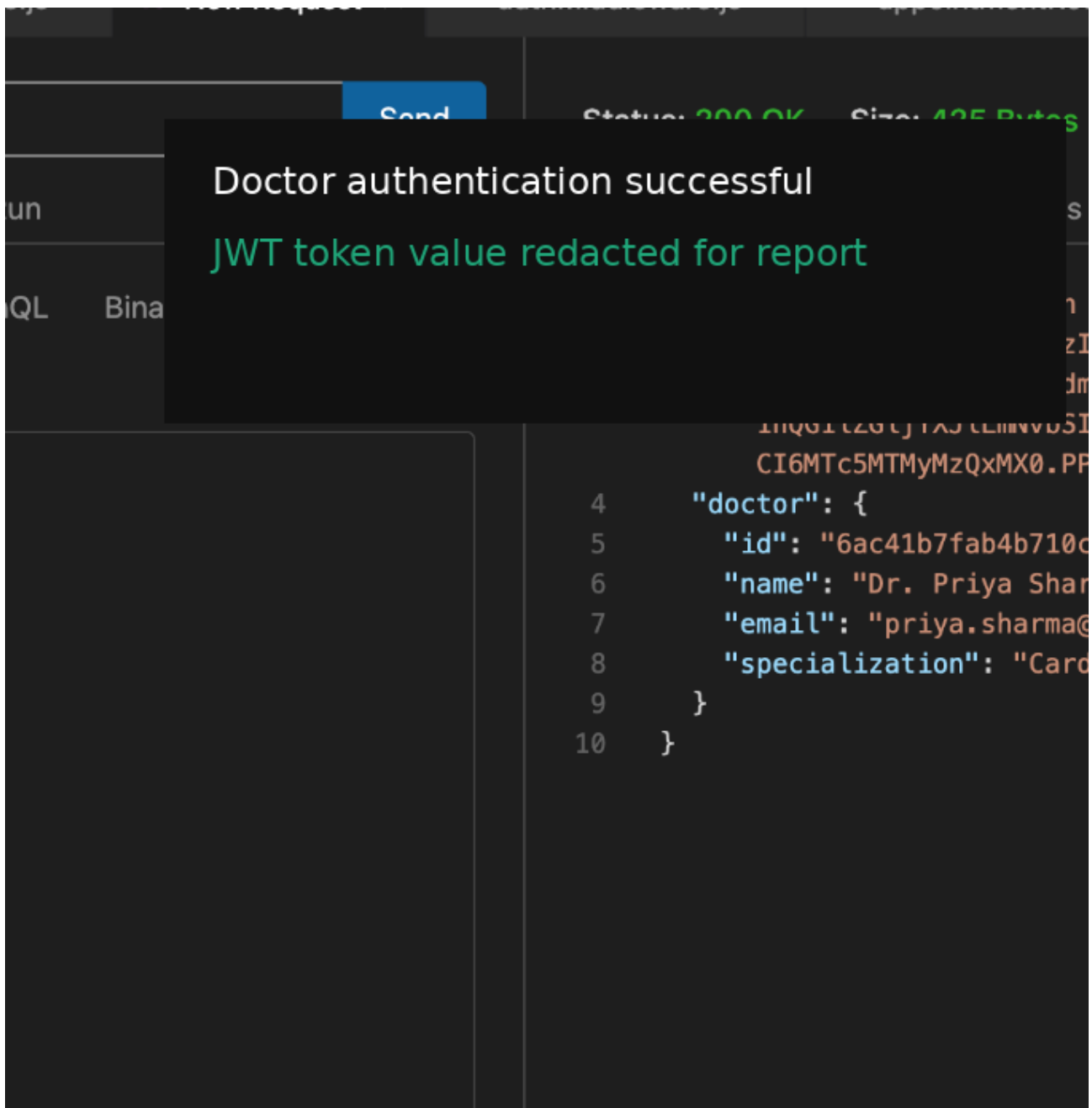


Figure 3: Project implementation and testing evidence.

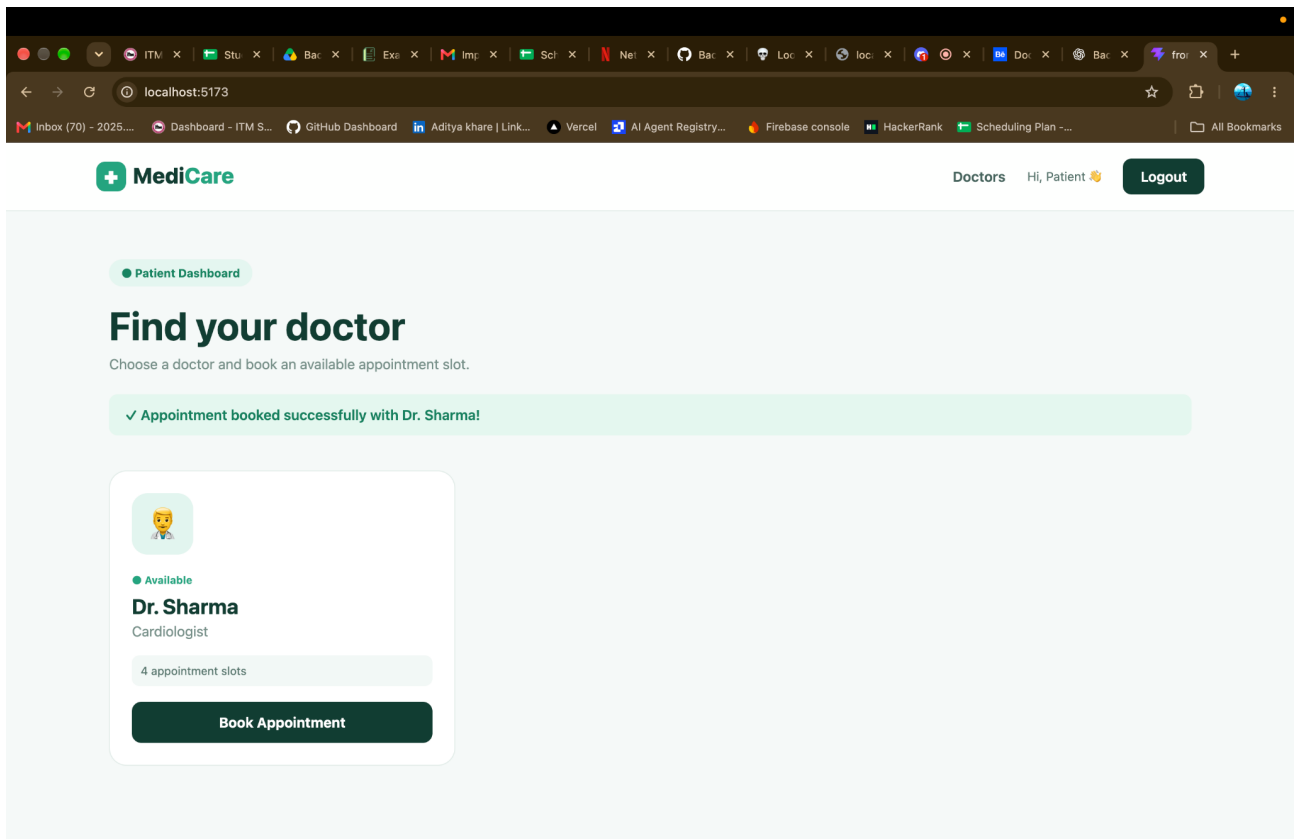


Figure 4: Project implementation and testing evidence.

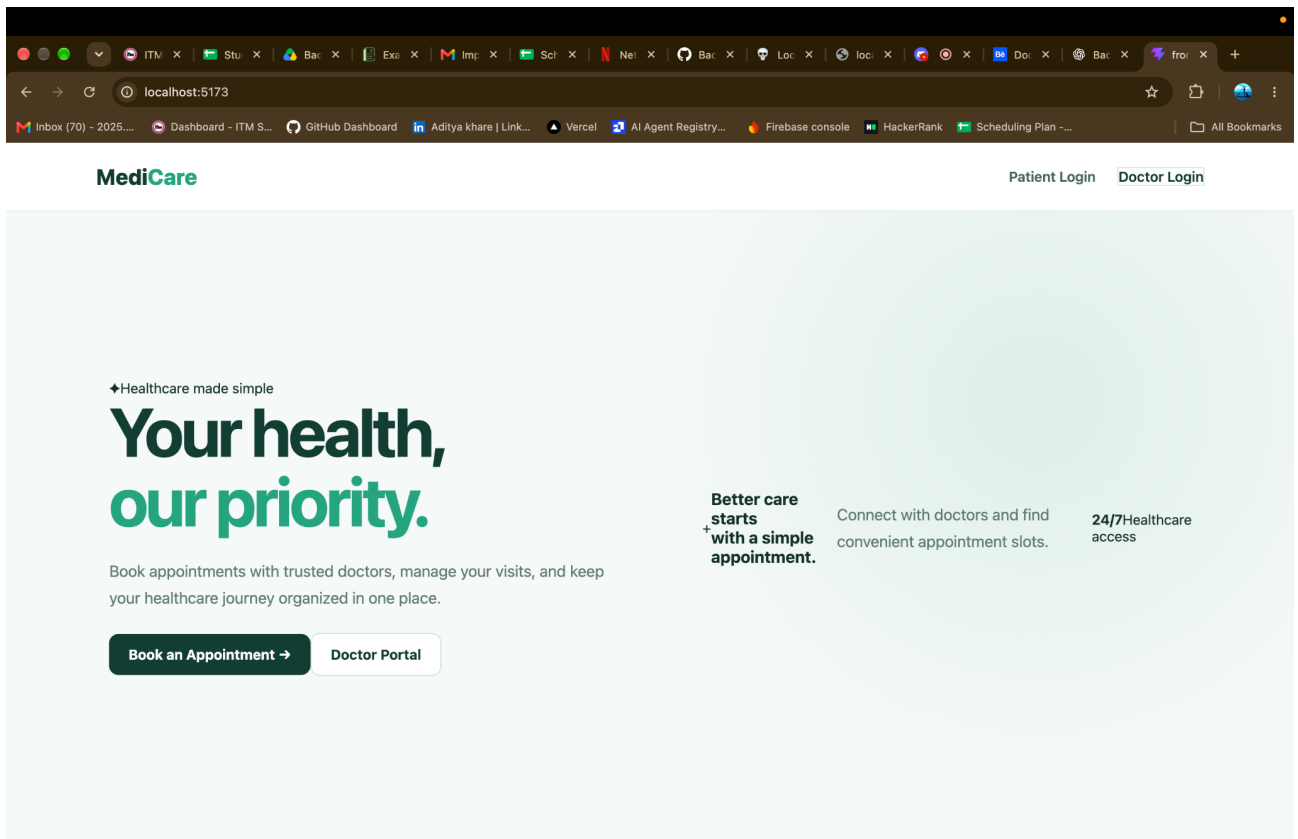


Figure 5: Project implementation and testing evidence.

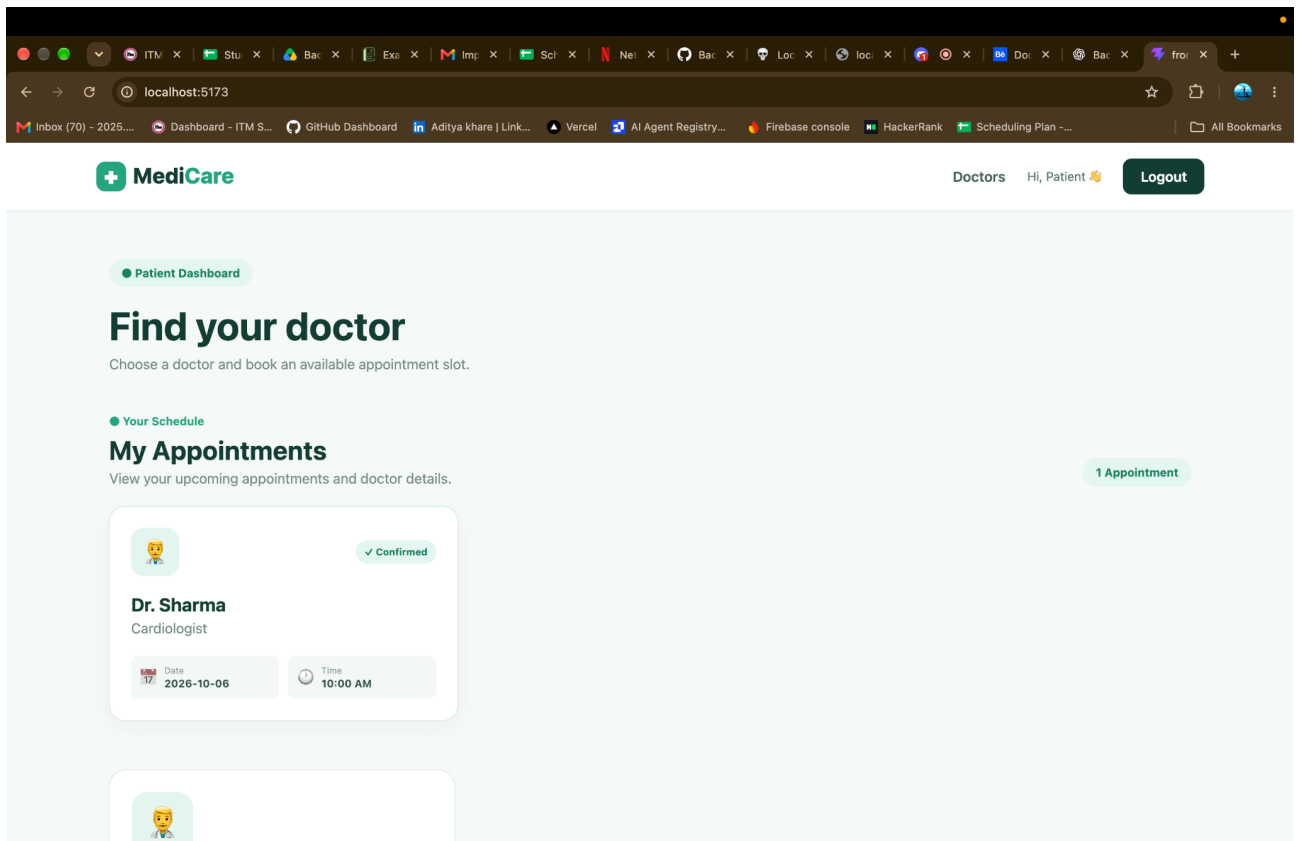


Figure 6: Project implementation and testing evidence.

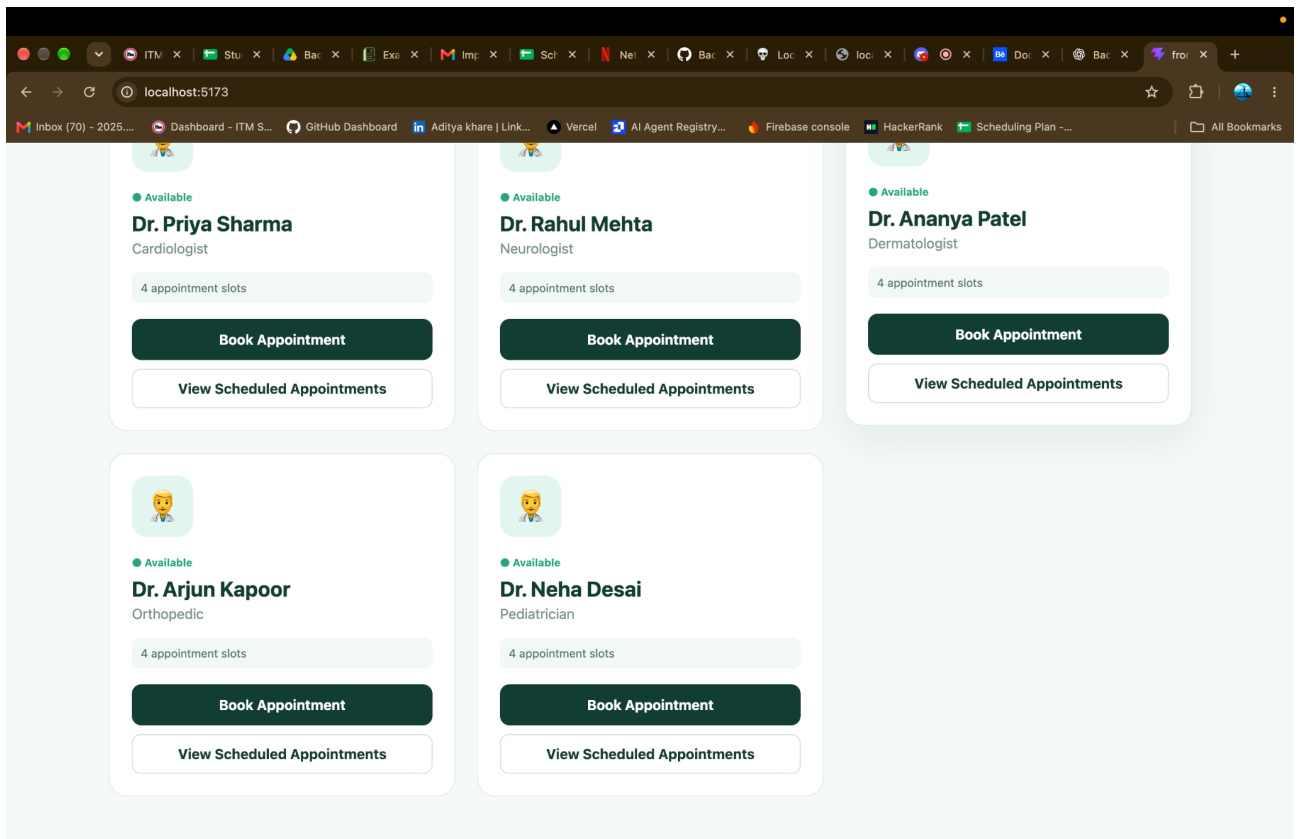


Figure 7: Project implementation and testing evidence.

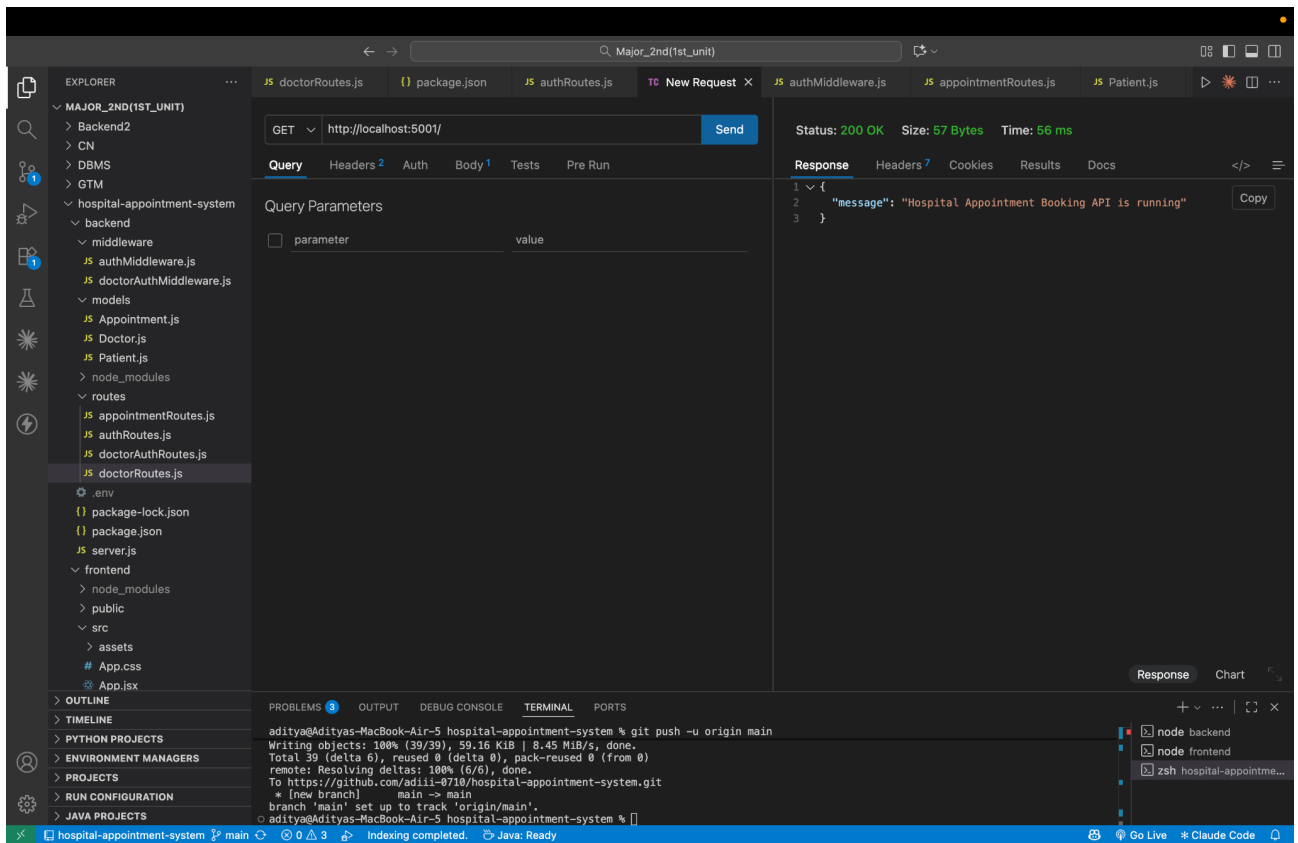


Figure 8: Project implementation and testing evidence.

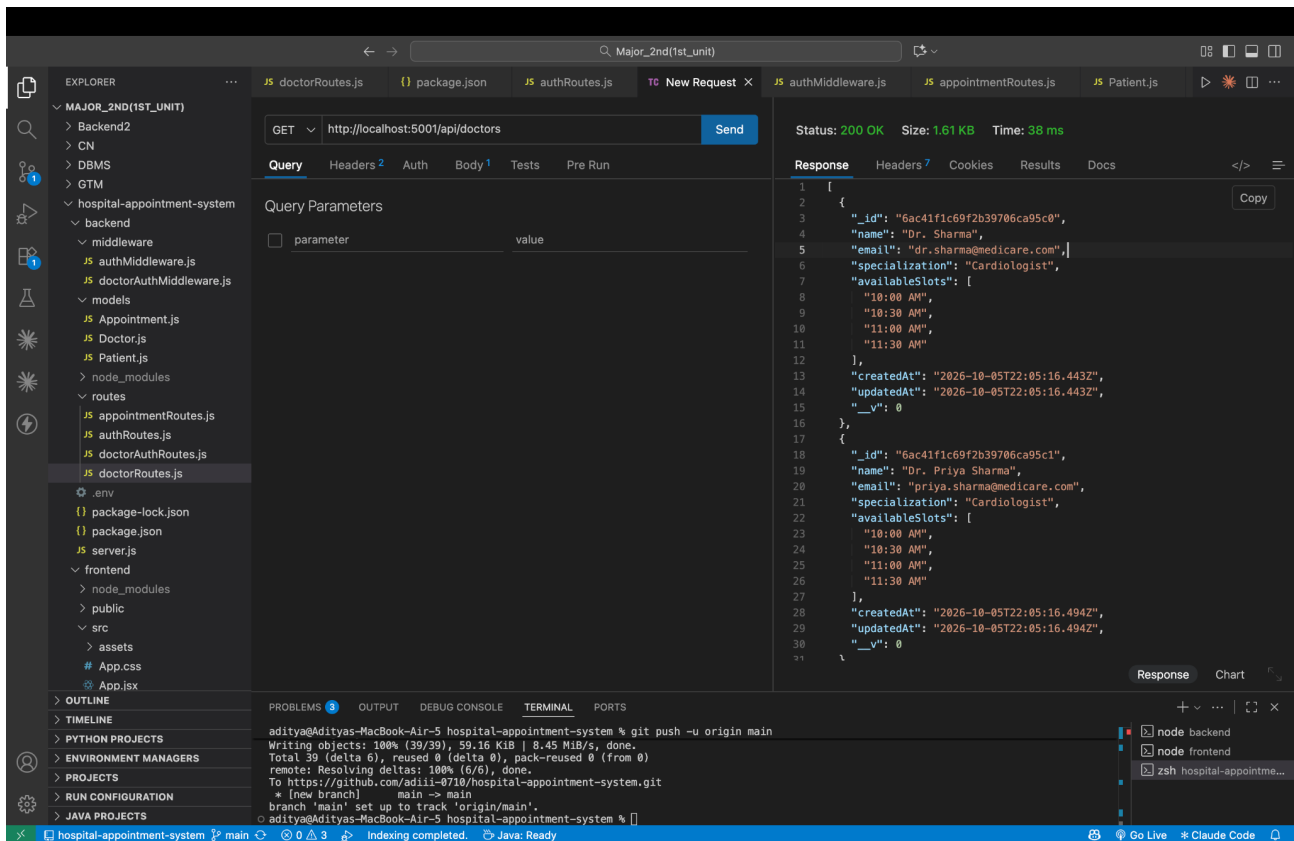


Figure 9: Project implementation and testing evidence.

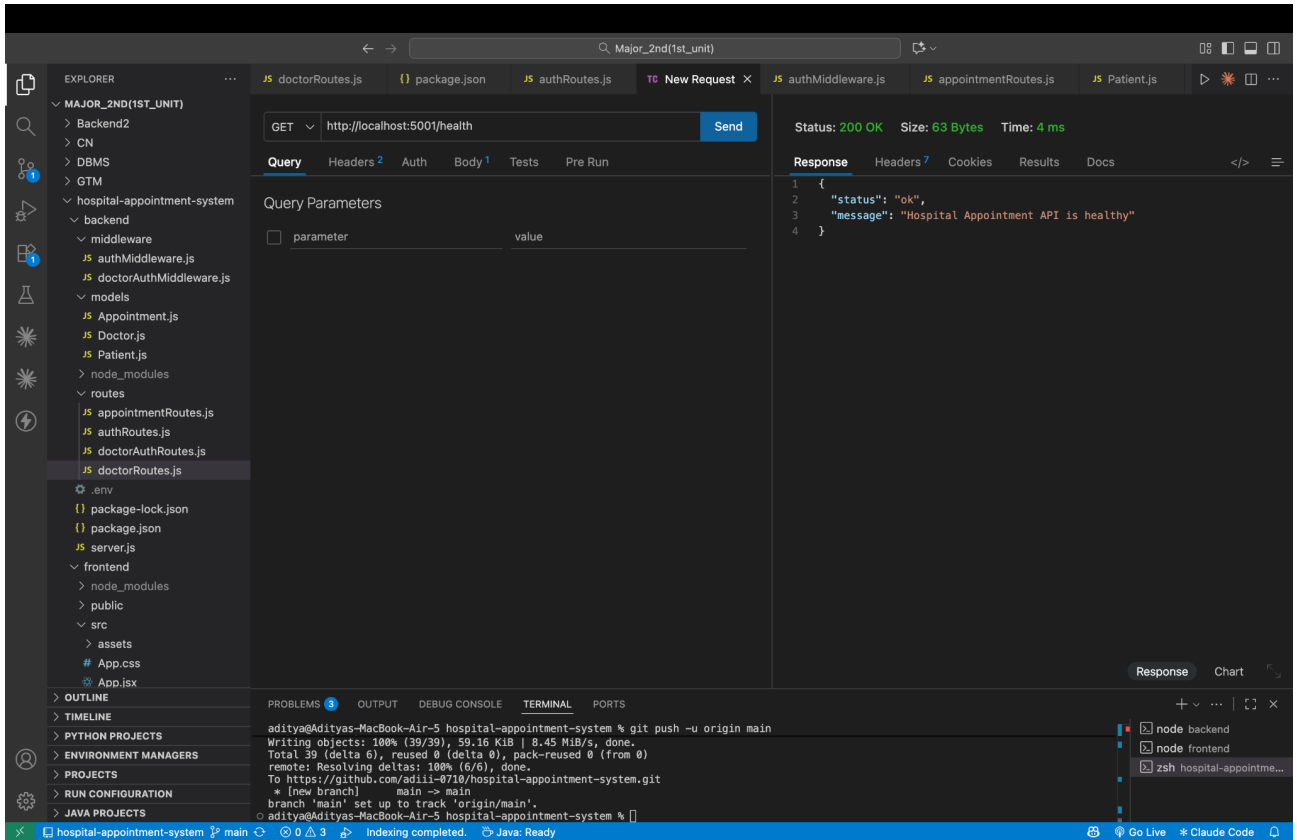


Figure 10: Project implementation and testing evidence.

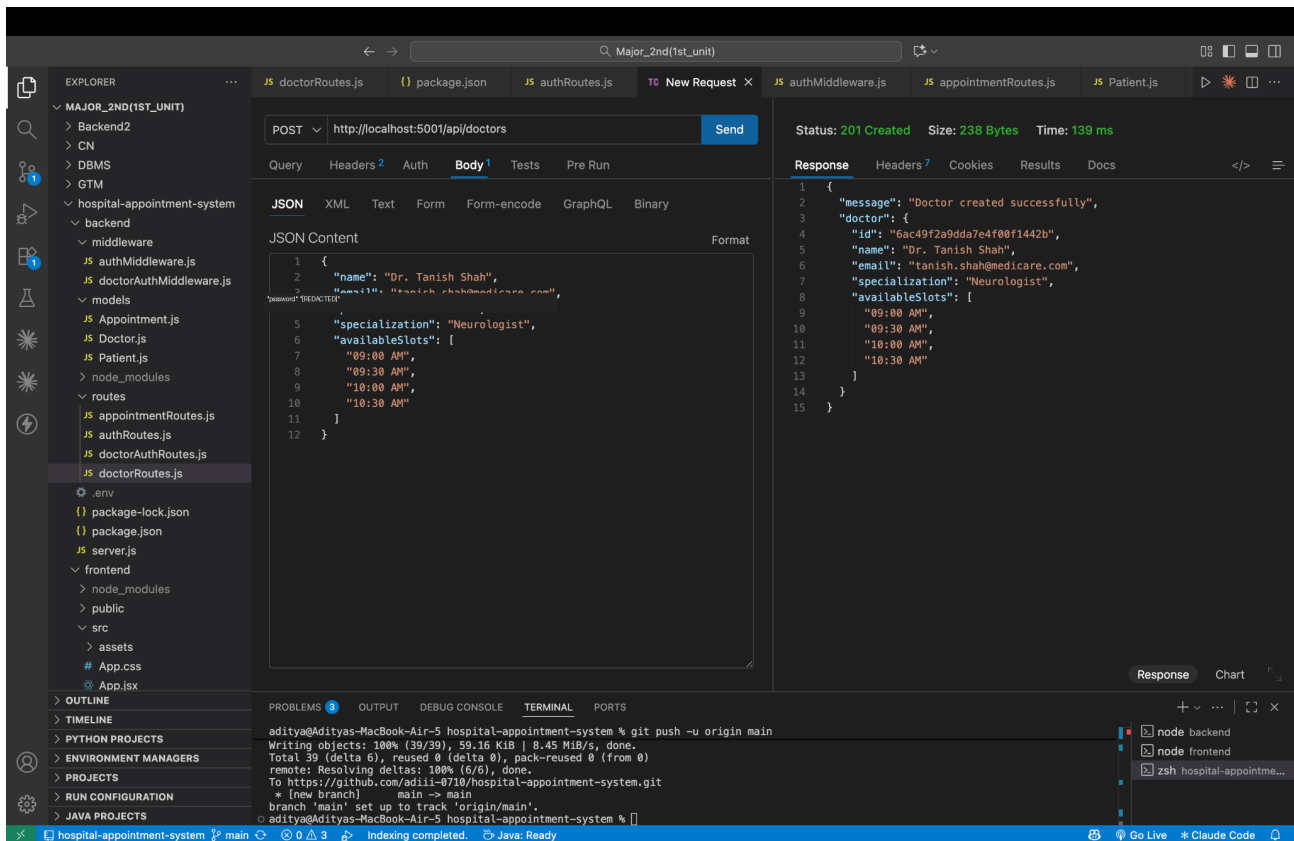


Figure 11: Project implementation and testing evidence.

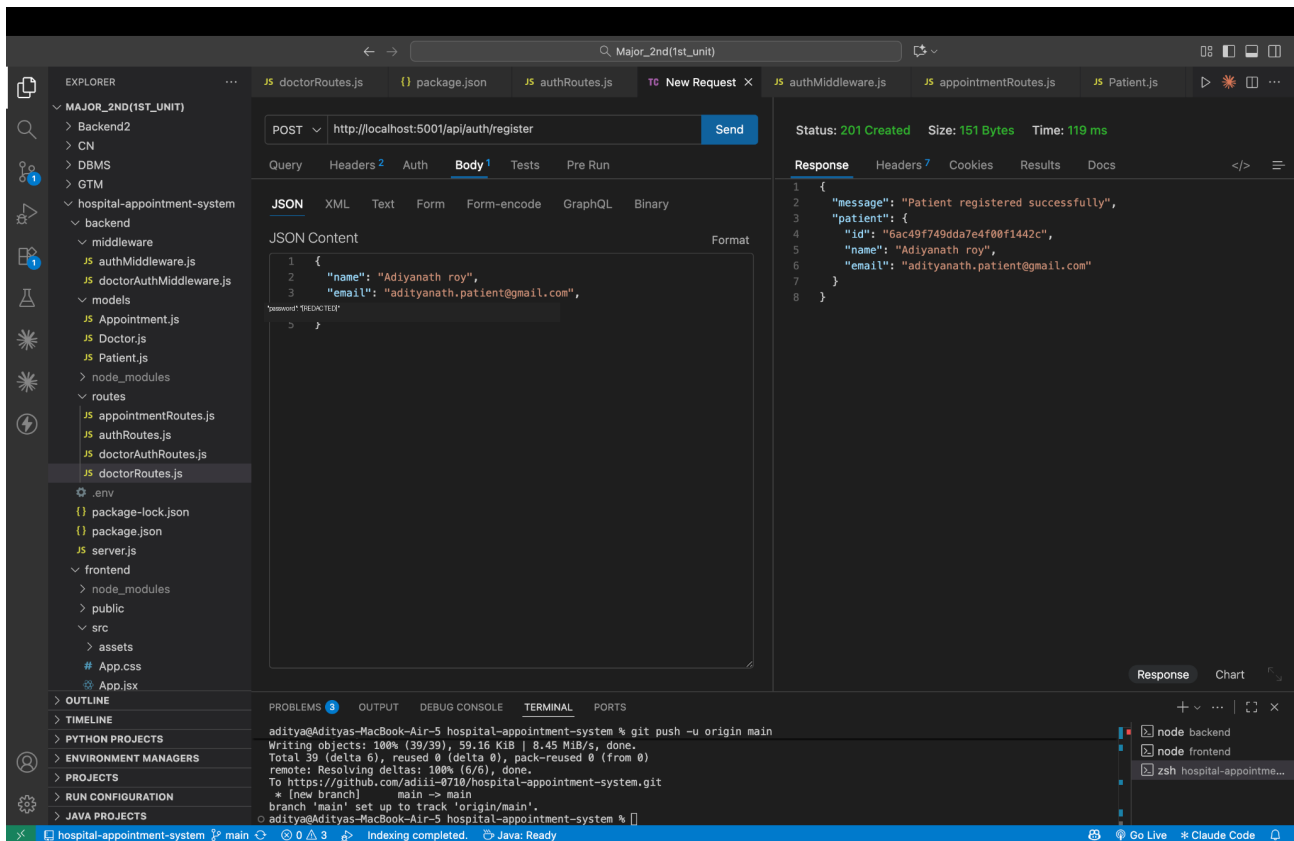


Figure 13: Project implementation and testing evidence.

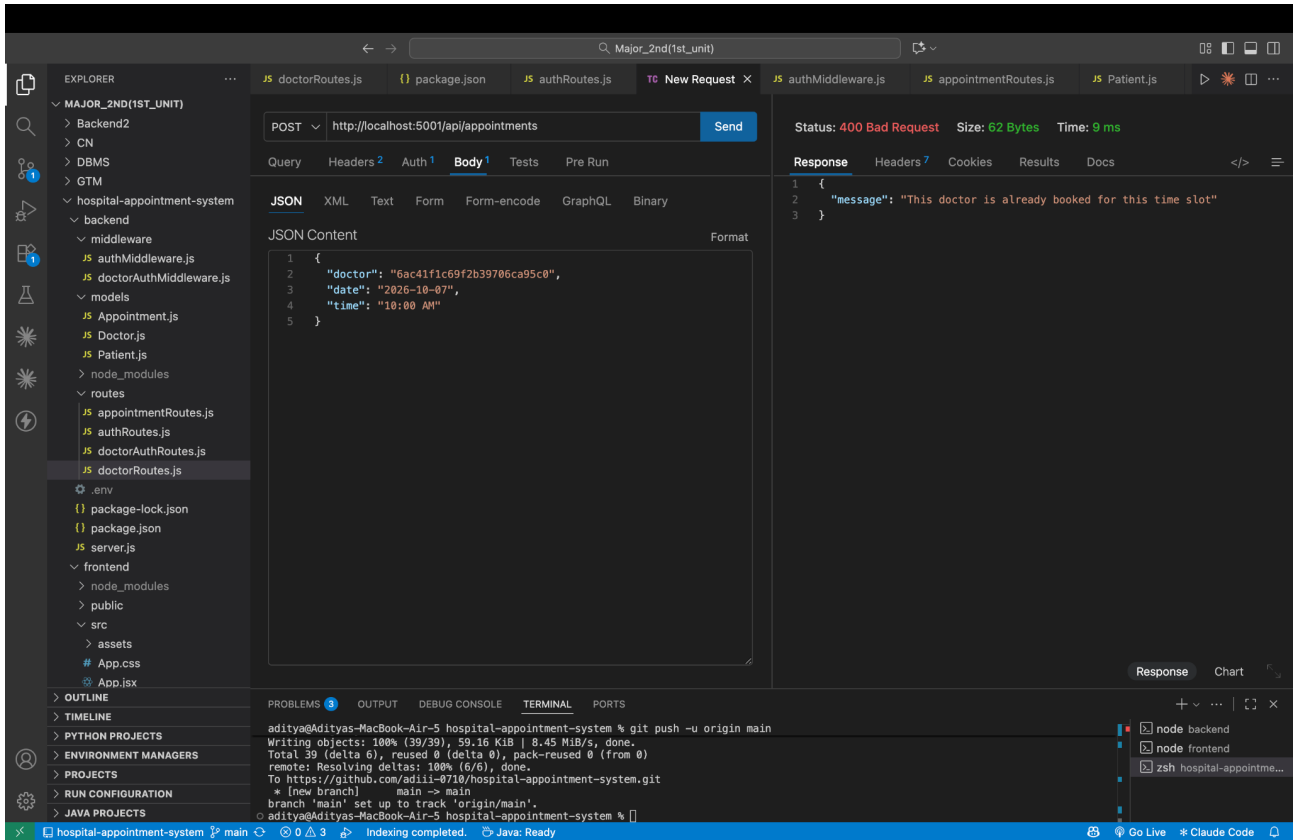


Figure 14: Project implementation and testing evidence.

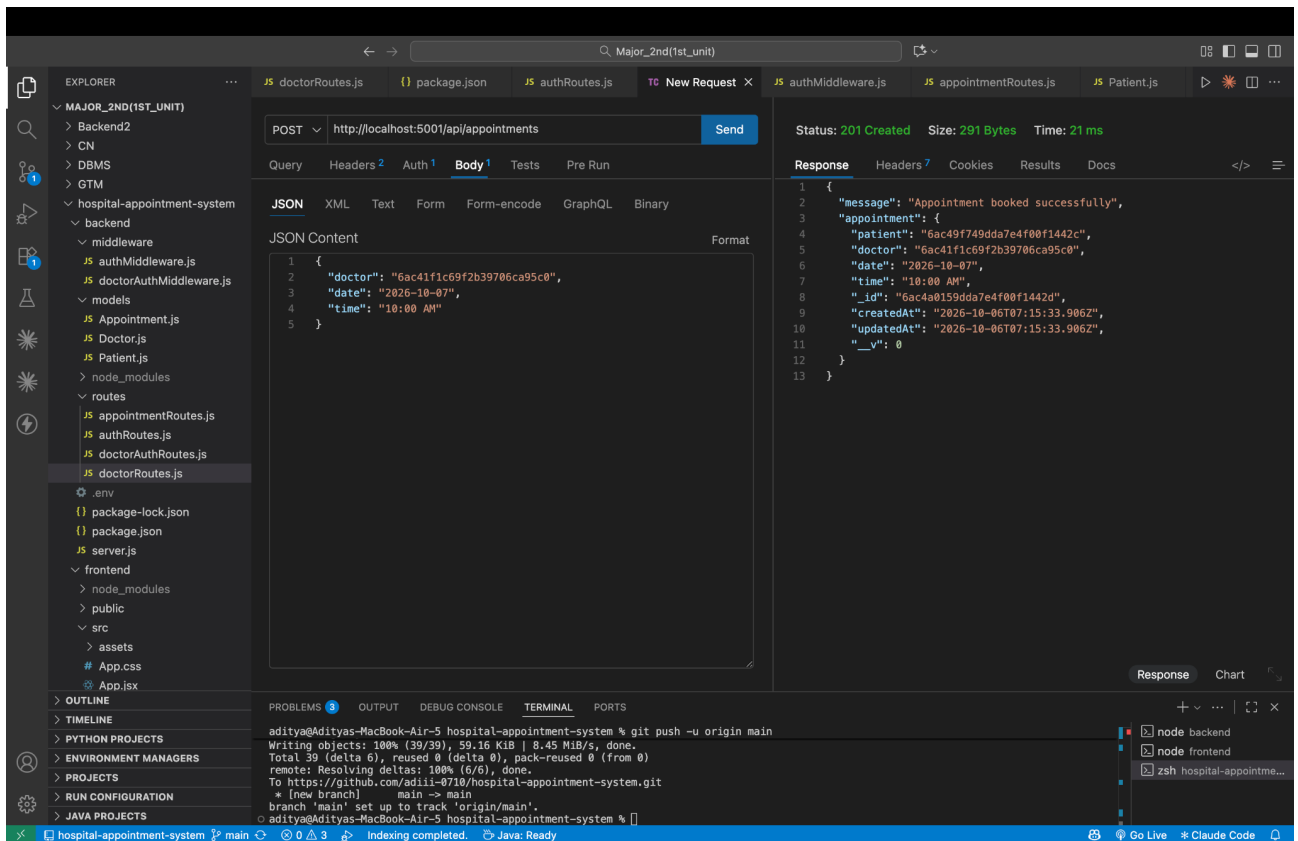


Figure 15: Project implementation and testing evidence.

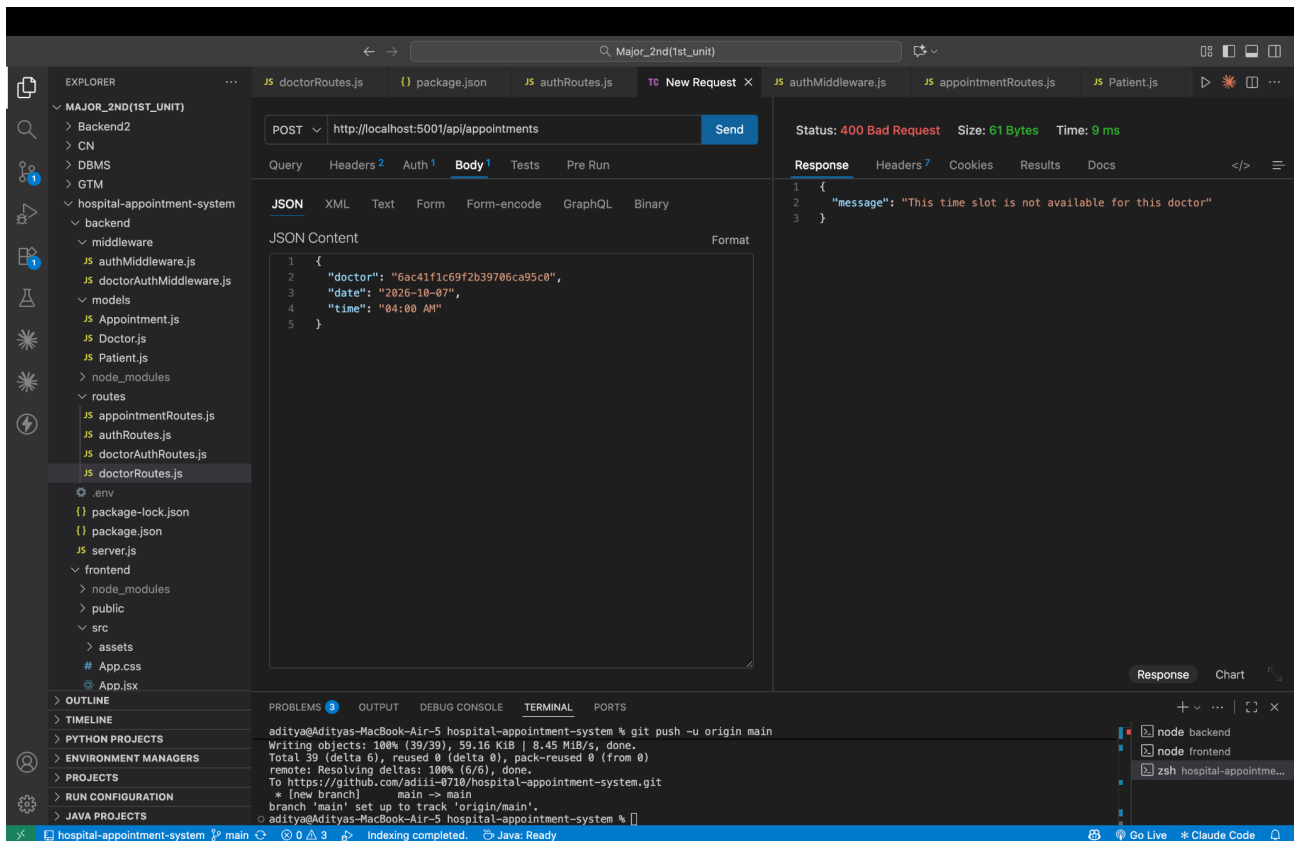


Figure 16: Project implementation and testing evidence.

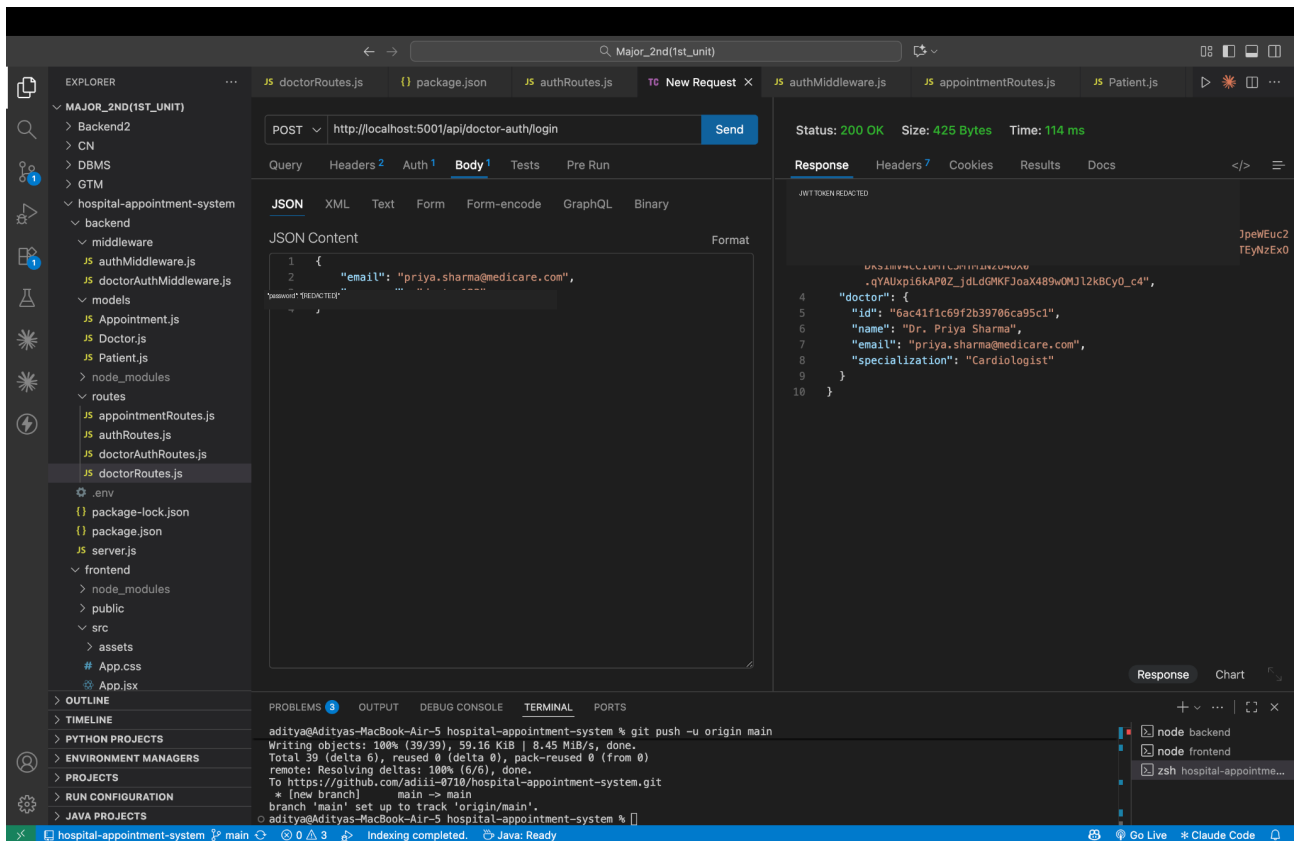


Figure 17: Project implementation and testing evidence.

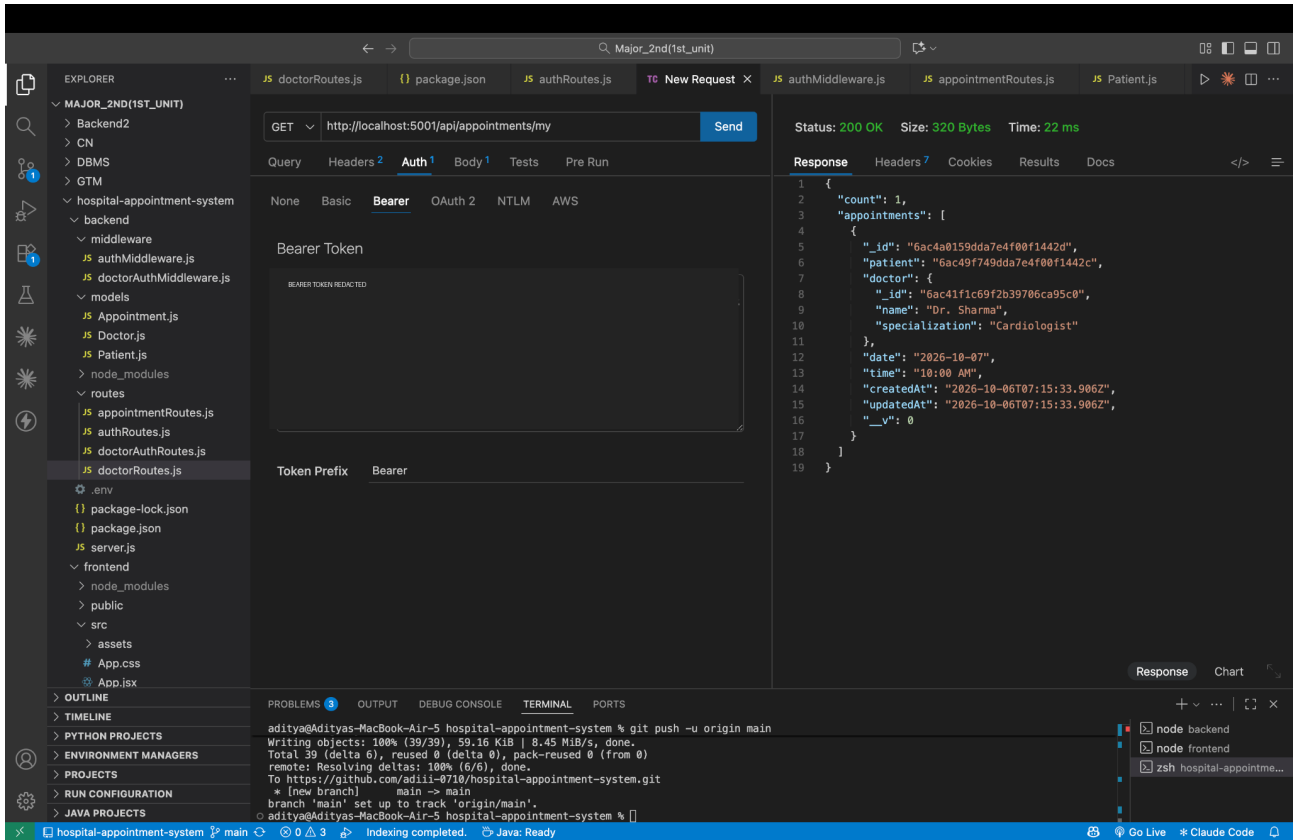


Figure 18: Project implementation and testing evidence.

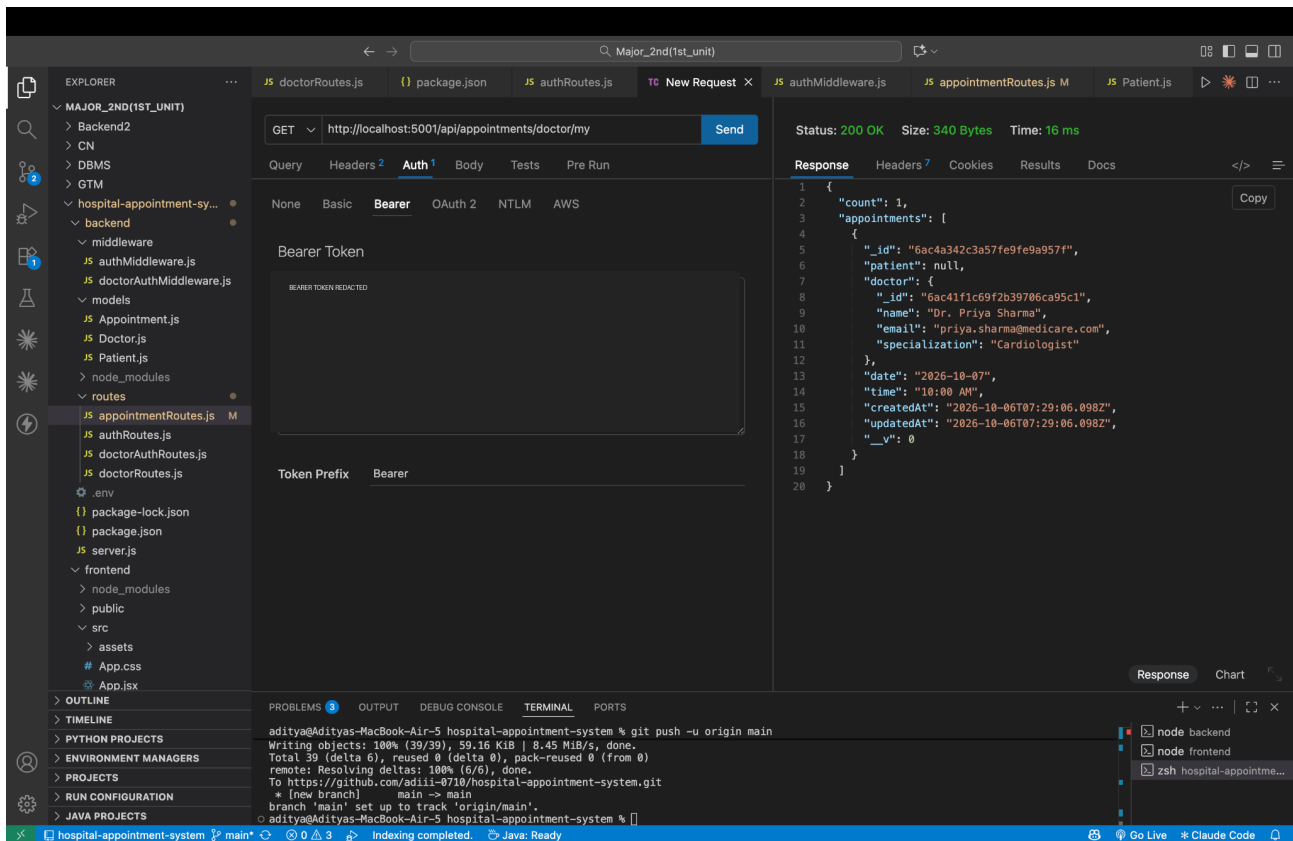


Figure 19: Project implementation and testing evidence.

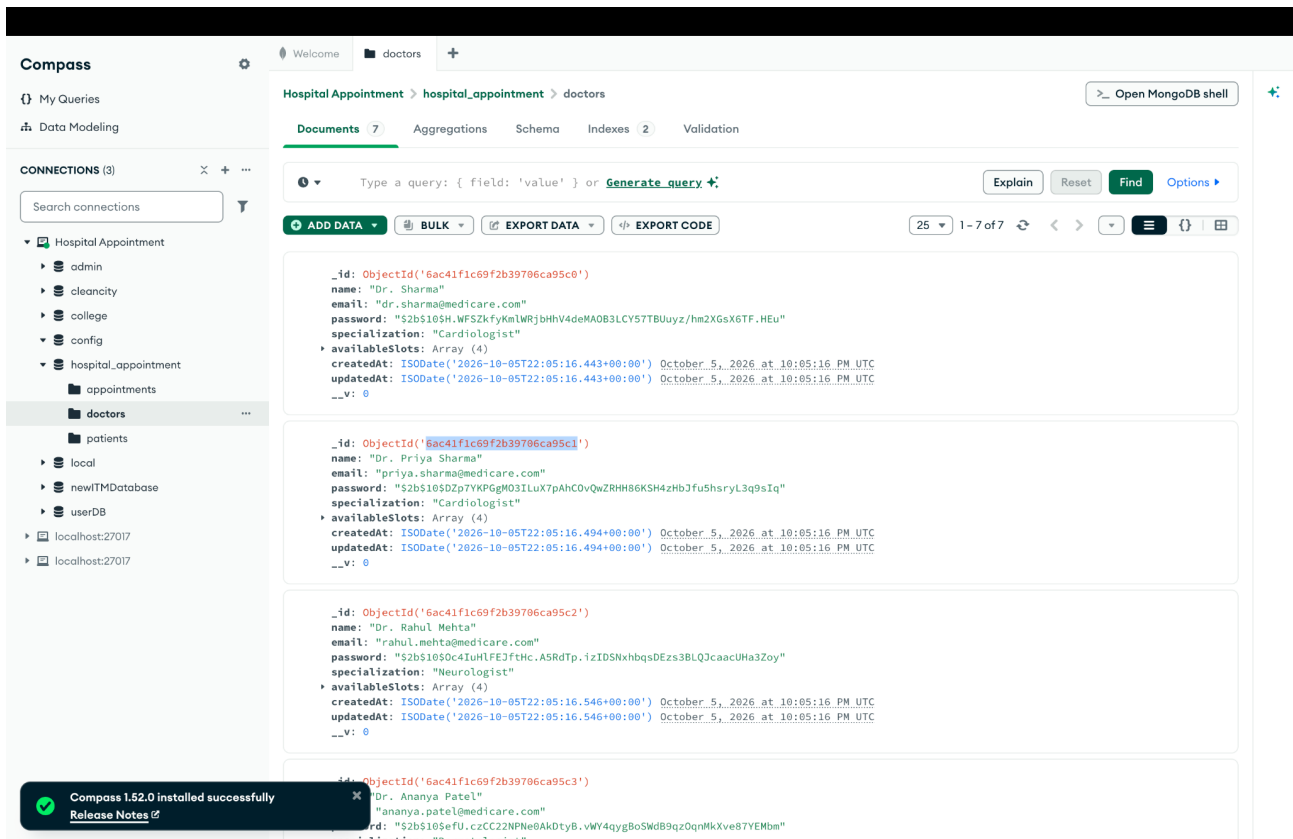


Figure 20: Project implementation and testing evidence.

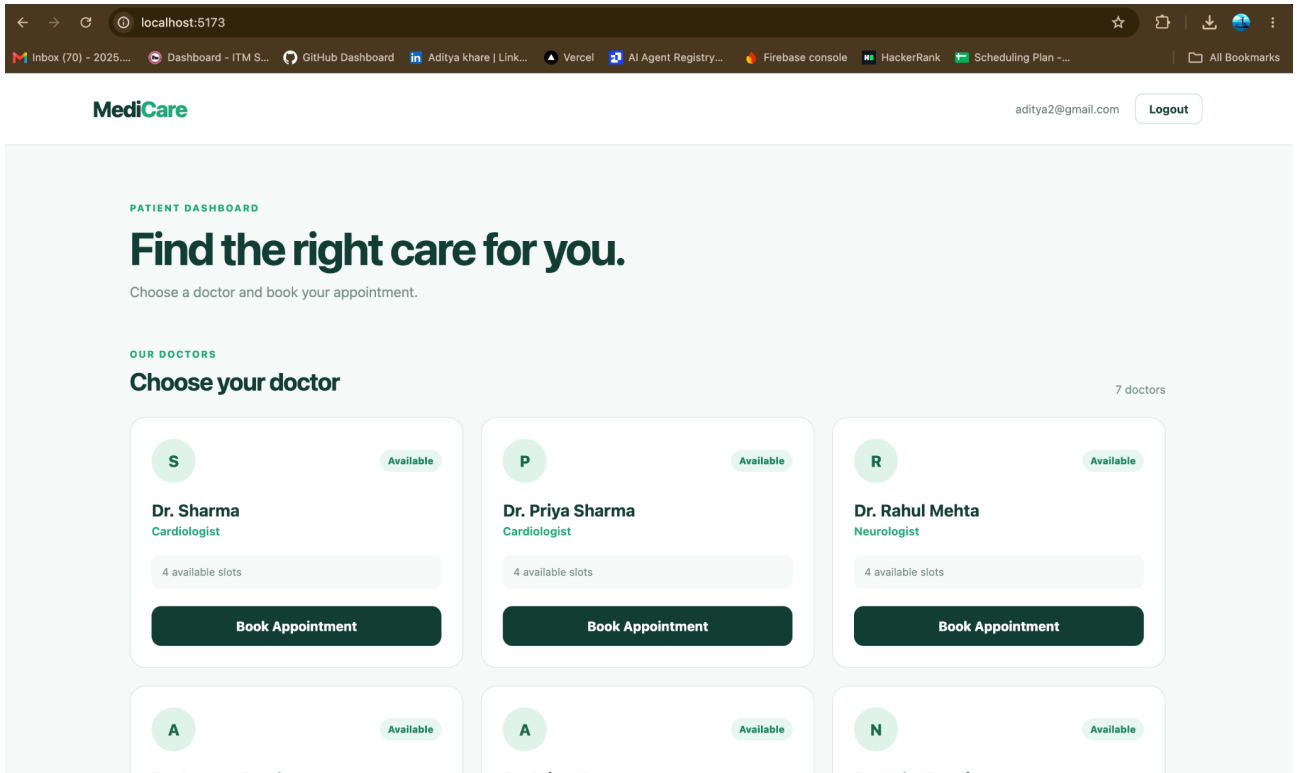


Figure 21: Project implementation and testing evidence.

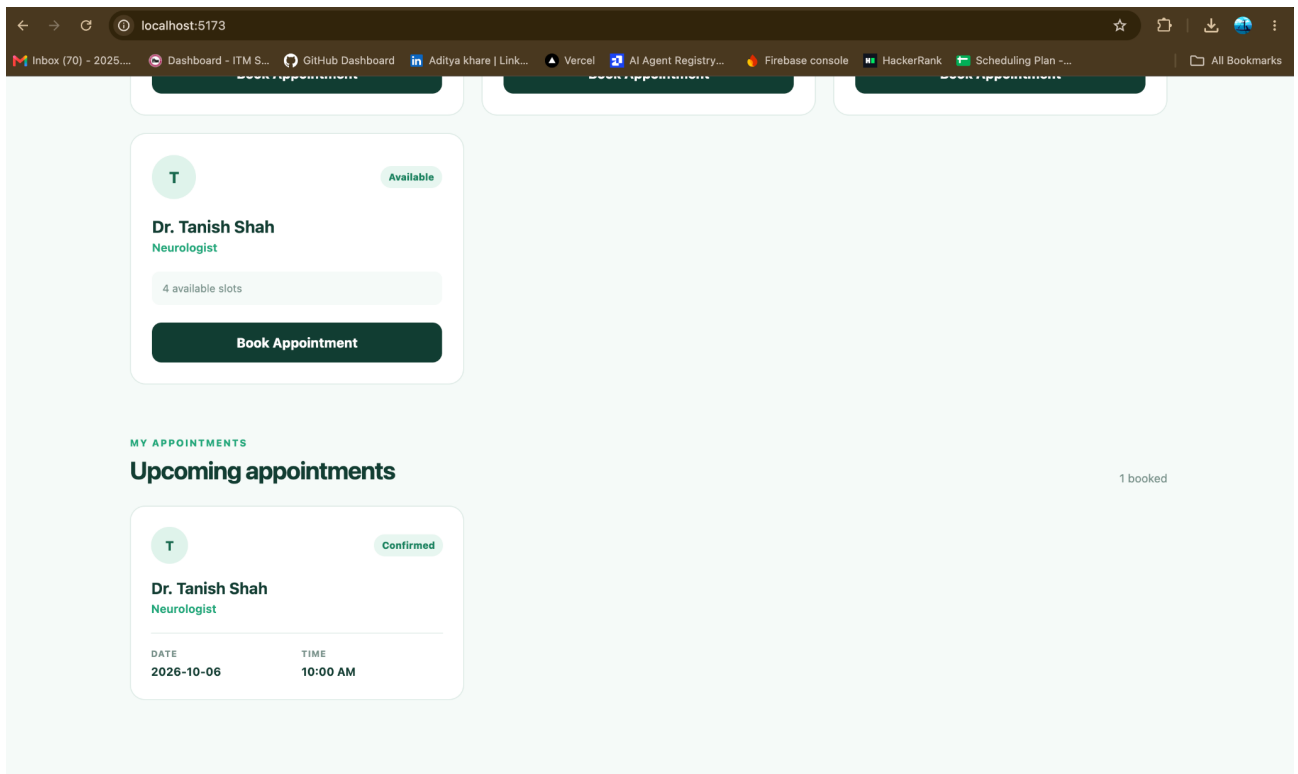


Figure 22: Project implementation and testing evidence.